DBT Day 5

Shivam Zanjurne · Tue, 13 Oct 26 · Subqueries (scalar → correlated, IN / ANY / ALL), DDL implicit commit, GRANT / REVOKE, Views (updatable vs not), ACID properties

---

Table of Contents

1. Warm-up: Rank vs Dense Rank, Delete vs Truncate
2. Setup: Department and Employee Tables
3. What Is a Subquery?
4. Scalar Subquery
5. Single-Row Subquery
6. Multi-Row Subquery
7. Multi-Column Subquery
8. Operators: IN, ANY, ALL
9. Correlated Subquery
10. TCL Recap and DDL Implicit Commit
11. DCL — Users, GRANT, REVOKE
12. Views — Definition, Types, Updatability
13. ACID Properties
14. Schema vs Database and InnoDB Storage (Brief)
15. Day Wrap and Practice Checklist

Warm-up: Rank vs Dense Rank, Delete vs Truncate

Class opened with a short practice reminder from Day 4.

Window functions again

`RANK`, `DENSE_RANK`, and `ROW_NUMBER` are **predefined** functions. You do **not** need their internal algorithm for exams — you need:

- **When** to use them (similar salaries / seniority / buckets)
- **What the output looks like** (ties skip or not)

Function	Same salary	Next rank
`ROW_NUMBER`	Different numbers	Always +1
`RANK`	Same number	**Skips**
`DENSE_RANK`	Same number	**No skip**

Run them again on `employee` and **read the result grid** until the difference is automatic.

Truncate cannot roll back (DDL)

Someone asked: after `START TRANSACTION`, can I `TRUNCATE` and then `ROLLBACK`?

No. `TRUNCATE` is a **DDL** command. In MySQL/InnoDB, DDL does an **implicit commit**. Changes before/with that DDL are saved; you cannot undo the truncate with rollback.

	DELETE	TRUNCATE
Category	DML	DDL
Rollback inside a transaction	Yes	**No**
Effect	Removes rows (can be filtered)	Empties table structure stays

DDL set to remember: **CREATE, ALTER, DROP, TRUNCATE** — all can trigger implicit commit (demo later in the day).

Setup: Department and Employee Tables

Class created a fresh database (e.g. `october_13`) and two related tables.

```
CREATE DATABASE october_13;
USE october_13;

CREATE TABLE department (
    department_id   INT PRIMARY KEY,
    department_name VARCHAR(30) NOT NULL
);

CREATE TABLE employee (
    emp_id         INT PRIMARY KEY,
    emp_name       VARCHAR(30) NOT NULL,
    salary         DECIMAL(10, 2),
    department_id  INT,
    manager_id     INT,
    CONSTRAINT fk_employee_department
        FOREIGN KEY (department_id)
        REFERENCES department(department_id)
);
```

Notes from class:

- Prefer naming FK constraints so you can drop them cleanly later.
- `manager_id` was stored for later ideas; it is **not** yet a formal self-FK in every demo.
- Spelling mistakes like `dpet_id` happened live — always rename with `ALTER TABLE … CHANGE` if needed so join columns match.

Sample departments

```sql
INSERT INTO department (department_id, department_name) VALUES
    (10, 'IT'),
    (20, 'Sales'),
    (30, 'HR'),
    (40, 'Research');
```

Sample employees (matches class narrative)

```sql
INSERT INTO employee (emp_id, emp_name, salary, department_id, manager_id) VALUES
    (1, 'Aniket',   60000, 10, 1),
    (2, 'Pranav',   50000, 20, NULL),
    (3, 'Omkar',    70000, 30, 2),
    (4, 'Abhishek', 65000, 10, 2),
    (5, 'Aditya',   80000, 20, 1),
    (6, 'Shivraj',  50000, 40, NULL),
    (7, 'Malkit',   45000, 30, NULL);
-- Later demos also insert a second Aniket (id 8) with salary 60000, dept 10
```

department_id	Who (examples)
10 IT	Aniket 60k, Abhishek 65k
20 Sales	Pranav 50k, Aditya 80k
30 HR	Omkar 70k, Malkit 45k
40 Research	Shivraj 50k

Keep this mental map — every subquery example below uses these numbers.

What Is a Subquery?

> A **subquery** is a **query written inside another SQL query**.

- The **inner** query runs first (in normal / non-correlated cases) and produces a value or set of values.
- The **outer** query uses that result in `WHERE`, `FROM`, `SELECT`, etc.

```sql
-- Outer: employees whose department_id appears in department table
SELECT *
FROM employee
WHERE department_id IN (
    SELECT department_id
    FROM department
);
```

Inner `SELECT` returns the list of department ids; outer keeps only employees whose `department_id` is in that list.

Why learn “types”?

In industry you will not open a checklist titled “use a scalar subquery now.” You will face a **requirement** (“people above company average,” “anyone richer than someone in Sales,” “above **their own** department average”). The **type** is just a name for the shape of the inner result:

Shape of inner result	Name
1 row × 1 column	Scalar
1 row × many columns	Single-row
Many rows × 1 column	Multi-row
Many rows × many columns	Multi-column
Inner mentions outer columns	Correlated

Joins can often solve the same problem. Prefer whatever is clearer and correct for the schema — class showed **71 tables** on a real project to make the point: nested queries become natural when tables multiply.

Scalar Subquery

Definition

A **scalar subquery** returns **exactly one value**: **one row and one column**.

Classic producers of one cell: `AVG`, `SUM`, `MAX`, `MIN`, `COUNT`.

Example — earners above company average

```sql
SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary)
    FROM employee
);
```

How to read it:

1. Inner query: `AVG(salary)` → one number (company average), e.g. ~60000-ish depending on rows.
2. Outer query: for each employee, keep the row if `salary >` that single number.
3. Result: people above the company-wide average (e.g. Omkar, Abhishek, Aditya…).

Piece	Role
Inner `AVG(salary)`	Scalar — one cell
Outer `WHERE salary > …`	Compare each row to that cell

If the inner query somehow returned **two** numbers, MySQL would error when used with `>` — scalar context demands **one** value.

Single-Row Subquery

Definition

A **single-row subquery** returns **one row**, possibly with **multiple columns**.

You compare the outer side using the **same number of columns** (tuple comparison).

Example — match Aniket's salary and department

```sql
SELECT emp_name, salary, department_id
FROM employee
WHERE (salary, department_id) = (
    SELECT salary, department_id
    FROM employee
    WHERE emp_name = 'Aniket'
);
```

Unfolded meaning (if Aniket has salary 60000 and department 10):

```sql
SELECT emp_name, salary, department_id
FROM employee
WHERE salary = 60000
  AND department_id = 10;
```

Same result shape for the simple case — the subquery form is the **pattern** you need when the comparison values come from another lookup, not hard-coded literals.

Critical rule

The **inner** query must return **exactly one row**.

```sql
-- FAILS if department 20 has more than one employee:
WHERE (salary, department_id) = (
    SELECT salary, department_id
    FROM employee
    WHERE department_id = 20
);
-- Error: Subquery returns more than 1 row
```

Class demo: after inserting a **second** Aniket with the same salary and department, even `WHERE emp_name = 'Aniket'` inside became multi-row → single-row comparison broke.

	Outer query	Inner query
May return many rows?	Yes	**No** — must be one row for `=` / tuple `=`
Columns compared	Must match count/order of inner columns	One row of N columns

> Exam tip: "Single-row subquery" ≠ "outer returns one row." It means the **subquery** returns one row.

Multi-Row Subquery

Definition

Inner query returns **multiple rows** (usually one column). Outer uses set operators like `IN`, `ANY`, `ALL`, or `EXISTS` — **not** a bare `=` against many rows.

Example — employees in IT or Sales

```sql
-- Inner alone: department ids for IT and Sales → 10 and 20
SELECT department_id
FROM department
WHERE department_name IN ('IT', 'Sales');

-- Full query
SELECT emp_name, salary, department_id
FROM employee
WHERE department_id IN (
    SELECT department_id
    FROM department
    WHERE department_name IN ('IT', 'Sales')
);
```

Reading:

1. Inner → `{10, 20}`
2. Outer → every employee whose `department_id` is in that set

You could hard-code `WHERE department_id IN (10, 20)`, but the subquery version stays correct if ids change and names stay the labels you care about.

Multi-Column Subquery

Definition

Inner query returns **two or more columns**, often **many rows** (e.g. from `GROUP BY`). Outer must compare the **same column list**.

Example — people who match a department's minimum salary

```sql
-- Inner: min salary per department
SELECT MIN(salary), department_id
FROM employee
GROUP BY department_id;

-- Outer: employees whose (salary, department_id) pair appears in that set
SELECT emp_name, salary, department_id
FROM employee
WHERE (salary, department_id) IN (
    SELECT MIN(salary), department_id
    FROM employee
    GROUP BY department_id
);
```

Why both columns?

Inner returns pairs like `(60000, 10)`, `(50000, 20)`, `(45000, 30)`, `(50000, 40)`. Matching only on salary would wrongly mix departments. Matching `(salary, department_id)` keeps "minimum earner **of that department**."

Rule	Detail
Columns in `WHERE (…)`	Same count and compatible types as `SELECT` list of subquery
Operator	Often `IN` for multi-row multi-column; `=` only if single row

Class also stressed: when writing complex multi-column comparisons with `>` on both salary **and** department id together, the logic gets weird — prefer clear business meaning (min/max per group) over forced inequality on ids.

Operators: IN, ANY, ALL

Used with **multi-row** (and sometimes multi-column) subqueries.

Operator	Meaning in plain English
IN	Value equals **any** value in the subquery result set
ANY	Comparison is true for **at least one** returned value
ALL	Comparison is true for **every** returned value

IN — membership

```
WHERE department_id IN (SELECT department_id FROM department WHERE …)
-- true if department_id equals one of the returned ids
```

Same idea for the multi-column `IN` example above.

ANY — “better than at least one”

```sql
-- Salaries in Sales (department 20): e.g. 50000 and 80000
SELECT salary
FROM employee
WHERE department_id = 20;

-- Who earns more than ANY of those Sales salaries?
SELECT emp_name, salary
FROM employee
WHERE salary > ANY (
    SELECT salary
    FROM employee
    WHERE department_id = 20
);
```

Mental model for `> ANY (50000, 80000)`:

- True if salary > 50000 **OR** salary > 80000
- Which simplifies to: salary > **minimum** of the set (here > 50000)

Walk-through:

Person	Salary	> 50k?	> 80k?	Keep? (`ANY`)
Aniket	60k	Yes	No	**Yes** (one true is enough)
Pranav	50k	No	No	No
Omkar	70k	Yes	No	**Yes**
Aditya	80k	Yes	No	**Yes** (80 > 50)
Malkit	45k	No	No	No

So **ANY** ≠ “greater than the maximum.” It means **OR** across comparisons.

Equivalent idea:

```
salary > ANY (set)  ≈  salary > MIN(set)   when using >
salary < ANY (set)  ≈  salary < MAX(set)   when using <
```

ALL — “better than every one”

```sql
SELECT emp_name, salary
FROM employee
WHERE salary > ALL (
    SELECT salary
    FROM employee
    WHERE department_id = 10
);
```

Suppose department 10 salaries are `60000, 65000, 60000`.

Mental model for `> ALL (60000, 65000, 60000)`:

- True only if salary > 60000 **AND** salary > 65000 **AND** salary > 60000
- Which simplifies to: salary > **maximum** of the set (here > 65000)

Walk-through:

Person	Salary	>60k?	>65k?	>60k?	Keep? (`ALL`)
Aniket	60k	No	—	—	No
Omkar	70k	Yes	Yes	Yes	**Yes**
Abhishek	65k	Yes	No	—	No
Aditya	80k	Yes	Yes	Yes	**Yes**

	ANY	ALL
Logic	OR — one success enough	AND — every comparison must succeed
For `>`	Effectively vs **MIN**	Effectively vs **MAX**
Empty set edge	Advanced topic — know the main OR/AND idea first	

Class demo with department 20 and `> ALL` returned empty because nobody earned **more than 80000** (the Sales max). Switching the inner filter to department 10 produced Omkar and Aditya.

Correlated Subquery

Definition

A **correlated subquery** refers to columns from the **outer** query. The inner query is **re-evaluated for each outer row** (conceptually), because it depends on that row's values.

Example — salary above own department average

```sql
SELECT e.emp_name, e.salary, e.department_id
FROM employee AS e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employee AS e2
    WHERE e2.department_id = e.department_id   -- outer column!
);
```

Why aliases `e` and `e2`? Same table twice — without aliases MySQL cannot tell which `department_id` you mean.

Reading for one outer row (Omkar, dept 30, 70k):

1. Outer row: Omkar, department 30.
2. Inner: average salary **only for department 30**.
3. Keep Omkar if 70k > that department average.

Repeat for every employee. Result: people above **their** department mean (not the company mean).

	Independent subquery	Correlated subquery
Inner uses outer columns?	No	**Yes**
Can run inner alone?	Usually yes	Usually **no** (missing outer context)
Typical use	Company-wide stats	Per-group / per-parent stats

> Practice tip: first write the scalar "above company average" query, then change the inner `WHERE` to match `e.department_id` — that single change is the jump from independent to correlated.

TCL Recap and DDL Implicit Commit

Normal DML inside a transaction

```
START TRANSACTION;

INSERT INTO employee (emp_id, emp_name, salary, department_id, manager_id)
VALUES (10, 'Satish', 85000, 30, 5);

-- Row visible now
SELECT * FROM employee WHERE emp_id = 10;

ROLLBACK;
-- Satish gone — insert undone
```

`COMMIT` would have made Satish permanent. `ROLLBACK` undoes uncommitted DML.

DDL forces an implicit commit

```
START TRANSACTION;

INSERT INTO employee (emp_id, emp_name, salary, department_id, manager_id)
VALUES (11, 'Vidya', 75000, 30, 5);

CREATE TABLE sample_demo (
    id   INT,
    name VARCHAR(30)
);

ROLLBACK;

SELECT * FROM employee WHERE emp_id = 11;
-- Vidya is STILL there — CREATE TABLE already committed the transaction
```

What happened?

1. Transaction started; Vidya inserted (pending).
2. `CREATE TABLE` ran → InnoDB does **implicit COMMIT** of everything so far.
3. Later `ROLLBACK` has nothing left from that insert to undo.
4. Vidya stays; `sample_demo` also remains (DDL itself is not rolled back).

Same idea for **ALTER**, **DROP**, **TRUNCATE**. Even a **SAVEPOINT** before the DDL does **not** protect you — after DDL, prior work is already committed.

Command type	Examples	Rollback after START TRANSACTION?
DML	INSERT, UPDATE, DELETE	Yes (until commit / implicit commit)
DDL	CREATE, ALTER, DROP, TRUNCATE	Triggers commit — cannot undo prior DML that way

Storage engine note: MySQL's common transactional engine is **InnoDB**. Implicit commit behaviour for DDL is an engine/server rule you must respect in scripts.

DCL — Users, GRANT, REVOKE

DCL (Data Control Language) controls **who** can do **what**. Main verbs: `GRANT`, `REVOKE`.

Root (admin) can see everything. Other users start with almost nothing until you grant privileges.

List users

```
SELECT user, host
FROM mysql.user;
```

Default-ish users include `root`, system accounts (`mysql.sys`, …). You will add your own.

Create a user

```
CREATE USER 'second_user'@'localhost'
IDENTIFIED BY 'welcome@123';
```

Piece	Meaning
`'second_user'`	Login name
`'localhost'`	Host allowed to connect (use server IP if DB is remote)
`IDENTIFIED BY`	Password

Login from another terminal:

```
mysql -u second_user -p
# password: welcome@123
```

Before any grants, `SHOW DATABASES` for that user usually shows only system schemas — **not** your `october_13`.

GRANT on one table

```
-- As root, while using the right mental model of database.table
GRANT SELECT, INSERT
ON october_13.employee
TO 'second_user'@'localhost';
```

Effects after the user reconnects / refreshes session:

- `october_13` becomes visible in `SHOW DATABASES`
- `SHOW TABLES` shows **only** `employee` (not `department`) if that is the only granted table
- `SELECT` / `INSERT` on `employee` work
- `UPDATE` / `DELETE` → **ERROR … command denied**

```
-- Still as second_user — will fail until UPDATE is granted
UPDATE employee
SET salary = 86000
WHERE emp_id = 3;
```

Add update:

```
GRANT UPDATE
ON october_13.employee
TO 'second_user'@'localhost';
```

See what a user has

```
SHOW GRANTS FOR 'second_user'@'localhost';
```

GRANT on whole database

```
GRANT ALL PRIVILEGES
ON october_13.*
TO 'second_user'@'localhost';
```

`database.*` = every table in that database. After re-login, `SHOW TABLES` can show `department` and `employee`; delete/drop/etc. become possible if included in `ALL`.

Global privileges

```
GRANT ALL PRIVILEGES
ON *.*
TO 'second_user'@'localhost';
```

`*.*` = **all databases, all tables** (global). Now the user can see and use databases across the instance (still subject to exact privilege set).

WITH GRANT OPTION

Without it: user may use privileges but **cannot** create users / grant those privileges onward.

```
GRANT ALL PRIVILEGES
ON *.*
TO 'second_user'@'localhost'
WITH GRANT OPTION;
```

Now `second_user` can:

```
CREATE USER 'third_user'@'localhost' IDENTIFIED BY 'welcome@123';

GRANT SELECT, INSERT, UPDATE
ON october_13.*
TO 'third_user'@'localhost';
```

Important rule: a user can only grant privileges they **themselves** hold (and only if they have GRANT OPTION). If `second_user` only received SELECT/INSERT/UPDATE on `october_13.*`, they cannot give `third_user` DELETE on that database.

> Exam tip from class: for exam accounts, admins often give `GRANT ALL PRIVILEGES ON *.* … WITH GRANT OPTION` so you can work freely.

Column-level privileges

You can restrict privileges to **named columns**:

```
-- SELECT only on emp_name and salary
GRANT SELECT (emp_name, salary)
ON october_13.employee
TO 'third_user'@'localhost';

-- INSERT only on emp_id and emp_name (salary not allowed)
GRANT INSERT (emp_id, emp_name)
ON october_13.employee
TO 'third_user'@'localhost';
```

Behaviour:

```sql
-- OK
SELECT emp_name, salary FROM employee;

-- Not OK (no privilege on other columns via *)
SELECT * FROM employee;   -- denied / incomplete depending on server checks

-- INSERT with salary → denied for column salary
INSERT INTO employee (emp_id, emp_name, salary)
VALUES (15, 'Sid', 60000);

-- INSERT only allowed columns → OK (other columns NULL if nullable)
INSERT INTO employee (emp_id, emp_name)
VALUES (15, 'Sid');
```

Syntax memory aid: list columns **in parentheses after the privilege**, not as `table.column` in the `ON` clause.

REVOKE

```sql
REVOKE SELECT, INSERT
ON october_13.employee
FROM 'third_user'@'localhost';

-- Or broader
REVOKE ALL PRIVILEGES
ON october_13.*
FROM 'second_user'@'localhost';
```

After revoke, re-login and `SHOW DATABASES` / `SHOW TABLES` shrink again.

Drop user

```sql
DROP USER 'third_user'@'localhost';
```

Include `@'localhost'` (or the host you used). Plain `DROP USER third_user` may fail depending on how the account was created.

Privilege levels (summary ladder)

```
Global:     GRANT … ON *.* …
Database:   GRANT … ON october_13.* …
Table:      GRANT … ON october_13.employee …
Column:     GRANT SELECT (col1, col2) ON october_13.employee …
```

Practice creating your **own** lab user with global privileges and working as that user — class recommended this from tomorrow onward.

Views — Definition, Types, Updatability

What is a view?

A **view** is a **named, saved SELECT**. It stores the **definition of the query**, **not** a private copy of the data.

	View	Temporary table
Stores	Query definition	Actual rows (materialized)
Lifetime	Until `DROP VIEW` (permanent object)	Session / until drop
Typical use	Reuse complex SELECTs cleanly	Scratch intermediate data

Why use views?

- Complicated joins/filters needed **many times**
- Write once: `CREATE VIEW … AS SELECT …`
- Later: `SELECT * FROM my_view` instead of pasting a long query
- Optional security: grant SELECT on a view that hides sensitive columns

Simple view

```sql
CREATE VIEW select_employee AS
SELECT emp_id, emp_name, salary, department_id
FROM employee;

SELECT * FROM select_employee;
```

MySQL re-runs the stored SELECT against **current** table data each time you query the view.

Filtered view

```sql
CREATE VIEW sales_employees AS
SELECT emp_id, emp_name, salary
FROM employee
WHERE department_id = 20;

SELECT * FROM sales_employees;
-- e.g. Pranav, Aditya — without rewriting the WHERE every time
```

Multi-table view (join)

```sql
CREATE VIEW sales_employee_with_department AS
SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    d.department_name
FROM employee AS e
JOIN department AS d
    ON e.department_id = d.department_id
WHERE e.department_id = 20;

SELECT * FROM sales_employee_with_department;
```

(Class also used old `FROM employee e, department d WHERE …` join style — prefer `JOIN … ON`.)

Views can be used **like tables** in outer queries / subqueries (`SELECT … FROM sales_employees WHERE salary > 60000`).

Updatable views — DML through the view

For many **simple** views (one base table, no aggregates, clear key columns), INSERT / UPDATE / DELETE on the view **change the underlying table**.

```sql
-- Insert through sales_employees view
INSERT INTO sales_employees (emp_id, emp_name, salary)
VALUES (16, 'Vipul', 74000);

SELECT * FROM employee WHERE emp_id = 16;
-- Row appears in base table; department_id may be NULL
-- if the view never exposed that column
```

```sql
UPDATE sales_employee_with_department
SET salary = 95000
WHERE emp_id = 2;

SELECT salary FROM employee WHERE emp_id = 2;
-- Pranav's salary updated in the real employee table
```

Takeaway: updating a view is not “editing a snapshot” — it edits **base data** (when the view is updatable).

Non-updatable views — aggregates / GROUP BY

If the view definition uses **aggregates** (`AVG` , `SUM` , `MAX` , `MIN` , `COUNT`) or collapses rows with `GROUP BY` , MySQL generally **rejects** INSERT/UPDATE/DELETE on that view — there is no single base row to map a change to.

```sql
CREATE VIEW average_salary_per_department AS
SELECT department_id, AVG(salary) AS average_salary
FROM employee
GROUP BY department_id;

SELECT * FROM average_salary_per_department;

-- FAILS
UPDATE average_salary_per_department
SET average_salary = 71000
WHERE department_id = 10;
-- Error: view is not updatable
```

View style	Usually updatable?
Simple / filtered single-table	Often **yes**
Multi-table (depends on keys/joins)	Sometimes; class showed UPDATE working on join view for salary
Aggregate / GROUP BY	**No**

Drop a view

```sql
DROP VIEW sales_employees;
```

Views are **not** temporary — they stay in the database until dropped. You can also `GRANT` / `REVOKE` privileges **on views** the same way as on tables (practice in lab).

ACID Properties

ACID is a set of four properties that make database **transactions** correct and reliable. Interview favourite: "Explain ACID with an example."

Letter	Property
A	Atomicity
C	Consistency
I	Isolation
D	Durability

Running example — bank transfer

- Amit has ₹10,000; Aniket has ₹5,000.
- Amit transfers ₹1,000 to Aniket.
- After success: Amit ₹9,000, Aniket ₹6,000.
- Never acceptable: Amit debited but Aniket not credited (money vanished).

Conceptual SQL (illustrative):

```
START TRANSACTION;

UPDATE account
SET balance = balance - 1000
WHERE user_name = 'Amit';

UPDATE account
SET balance = balance + 1000
WHERE user_name = 'Aniket';

-- If BOTH succeeded:
COMMIT;
-- If ANY failed:
-- ROLLBACK;
```

A — Atomicity

> All operations in one transaction complete as **one unit**, or **none** of them remain (rollback).

All or nothing. Debit **and** credit both happen, or the whole transfer is undone.

C — Consistency

> A transaction moves the database from one **valid** state to another **valid** state, respecting rules/constraints.

Example: Amit has only ₹700 but tries to send ₹1,000. Balance must not go negative. Enforce with constraints:

```
-- Conceptual
CHECK (balance ≥ 0)
```

Primary key, foreign key, NOT NULL, UNIQUE, CHECK — these **rules** are how consistency is preserved.

I — Isolation

> Concurrent transactions are controlled so they do not corrupt each other's results.

Examples: **RedBus / BookMyShow** — one seat left, many users clicking Book. Only **one** booking should win.

Isolation does **not** mean "only one transaction in the whole database." Many transactions run together; the engine uses **locks** (e.g. InnoDB **row-level locking**) and concurrency control so results stay correct.

D — Durability

> After a successful **COMMIT**, changes **survive** crashes and restarts.

If the transfer commits and then the server dies, after restart Amit should still be at ₹9,000 and Aniket at ₹6,000. Committed work is **final**.

One-line revision

Property	One line
Atomicity	All or nothing
Consistency	Rules/constraints always hold
Isolation	Concurrent users don't break each other
Durability	Committed data stays after crash

Schema vs Database and InnoDB Storage (Brief)

Schema ≈ database in MySQL

In MySQL, `CREATE DATABASE` and `CREATE SCHEMA` are effectively the **same**. A schema/database holds tables, views, and later objects like triggers/procedures.

How InnoDB stores rows (class overview)

Workbench shows neat row grids; on disk InnoDB organizes data in **pages** (often **16 KB** max page size in common configs).

Area	Role
Buffer pool (in RAM)	Cache of frequently used pages
Disk	Permanent storage of pages
Overflow pages	Extra pages for large `TEXT` / `BLOB` values

You do not need every internal detail for Day 5 exams, but remember: engines cache pages in memory and persist them on disk — this ties to **durability** and performance (indexes come next class).

Deferred to next day

Indexes (with a speed demo: with vs without index) and **stored procedures** were postponed — class asked everyone to practice today's subquery / grant / view / ACID material first.

Day Wrap and Practice Checklist

Subquery cheat-sheet

Type	Inner returns	Typical operators
Scalar	1×1	`>`, `<`, `=`, …
Single-row	1×N	`=` / tuple `=`
Multi-row	M×1	`IN`, `ANY`, `ALL`
Multi-column	M×N	`IN` on tuples
Correlated	Depends on outer row	Same, plus outer refs

Operator	Think
`IN`	Equal to one of the set
`> ANY`	Greater than at least one ≈ > MIN
`> ALL`	Greater than every one ≈ > MAX

Security cheat-sheet

```
CREATE USER 'u'@'localhost' IDENTIFIED BY 'pwd';
GRANT … ON db.table / db.* / *.* TO 'u'@'localhost' [WITH GRANT OPTION];
SHOW GRANTS FOR 'u'@'localhost';
REVOKE … ON … FROM 'u'@'localhost';
DROP USER 'u'@'localhost';
```

Views cheat-sheet

- Stores **definition**, not a data dump
- Simple / filtered / multi-table for reuse

- Often updatable if simple; **not** updatable with aggregates
- `DROP VIEW name`

ACID

Atomicity · Consistency · Isolation · Durability — explain with the bank transfer story.

Practice (do this in lab)

1. Rebuild `department` / `employee`; run scalar, single-row, multi-row, multi-column, ANY, ALL, correlated examples; predict results before looking.
2. `START TRANSACTION` → INSERT → `CREATE TABLE` → `ROLLBACK` → prove the insert stayed.
3. Create `lab_user`, grant table → database → column → global; try denied UPDATE; revoke; drop user.
4. Create filtered + join views; INSERT/UPDATE through them; create aggregate view and show UPDATE fails.
5. Write ACID answers in your own words with the Amit → Aniket ₹1000 story.

> Subqueries + GRANT/REVOKE + Views + ACID are **core interview / exam** topics. Class examples were simple on purpose — focus on **how each construct behaves**, then map them to whatever schema you meet later.
> Coverage: DBT Day 5 lecture (13 Oct 2026) — expanded from the Hinglish transcript into step-by-step English notes with runnable MySQL patterns. Indexes deferred to Day 6 as in class.